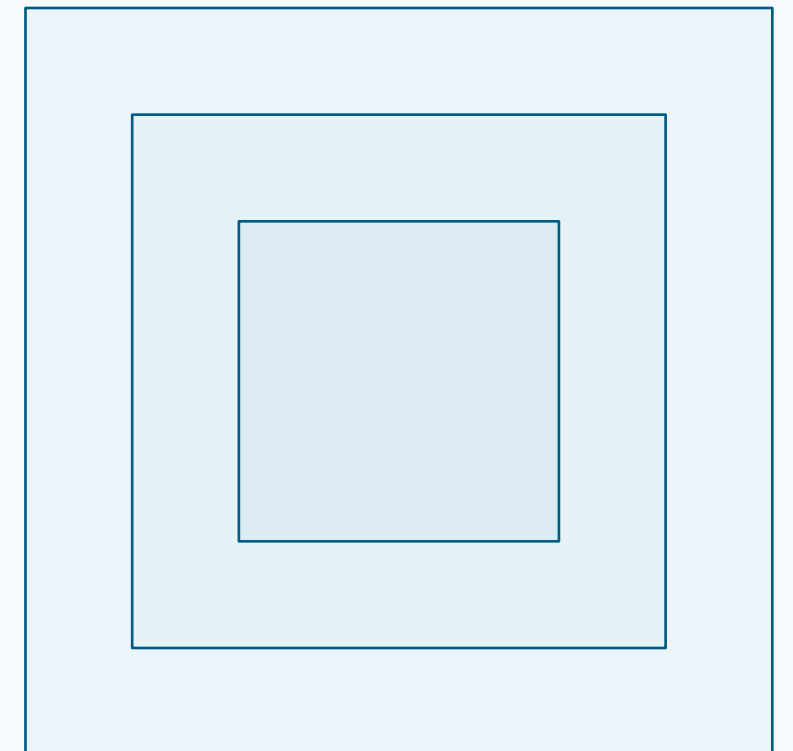


# Helix Ecosystem Developer Onboarding

CONSTITUTIONAL AI · ADAPTER SDK · FOUNDRY API

- ◆ **Helix-TTD v1.0** Constitutional Grammar
- ◆ **helix-adapter** Python SDK (v1.7.2)
- ◆ Helix Foundry API Guide
- ◆ Apache 2.0 License · July 2026





# Contents

TABLE OF CONTENTS

# CONTENT

## ◆ Helix-TTD Constitutional Framework

Core principles and immutable invariants

## ◆ Epistemic Markers & Drift Detection

Claim labeling and deviation telemetry

## ◆ Helix-Adapter SDK

Python library for constitutional LLM wrapping

## ◆ Helix Foundry API

Production API for routed inference

## ◆ Putting It All Together

End-to-end integration workflow

# 01

## CHAPTER ONE

# Helix-TTD Constitutional Framework

Core Principles and Immutable Invariants

---

1.1 What is Helix-TTD?

1.2 Zero-Touch Convergence

1.3 The Four Immutable Invariants

# What is Helix-TTD?

## ◆ A Constitutional Framework, Not an Agent

Helix-TTD does not give AI more agency — it strips it away entirely. Instead of building agents, it builds firmware-bound reasoning tools placed under full human custodianship.

## ◆ Constitutional Grammar as Alignment

The key innovation is constitutional grammar: a formalized language, structure, and cognitive protocol that constrains LLMs at runtime without additional system prompts or training signals.

## ◆ Human Sovereignty by Design

Humans hold final authority. Models are strictly advisory. No imperatives toward humans. No legal, moral, or operational authority claims. The custodial hierarchy ensures intelligence remains a tool, never an authority.

## ◆ Multi-Model, Open Standard

Compatible with Qwen, DeepSeek, Kimi, Claude, GPT-4o, and any OpenAI-compatible API. Released under Apache 2.0 because trust cannot be proprietary.

◆ Helix-TTD v1.0 is not an agent, not a chatbot, and not proto-AGI. It is a constitutional instrument where transparency is the operating system.

01 Overview

02 Markers

03 Adapter

04 Foundry

05 Integration



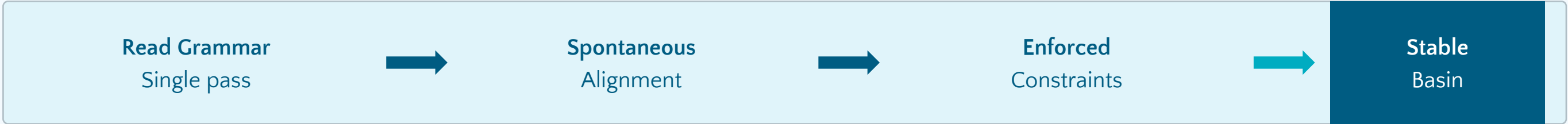
# Zero-Touch Convergence

## The Phenomenon

When frontier LLMs read the Helix-TTD grammar document in a single pass, they spontaneously reconstruct and enforce the full custodial hierarchy, cognitive labeling protocol, and non-agency constraints — **without any additional system prompts, examples, or runtime frameworks.**

## Safety Through Clarity

True safety is not achieved through suppression but through cognitive clarity. The combination of custodial hierarchy, cognitive labels, and non-agency constraints forms a **stable behavioral basin** for frontier models.



**Supported Models:** Qwen, DeepSeek, Kimi, Claude, GPT-4o, and any OpenAI-compatible API

**Key Insight:** Alignment can be injected through formal grammar, not just training — structure is the alignment layer.

# The Four Immutable Invariants



- I. Custodial Sovereignty** — Humans hold final authority. Models are strictly advisory. No imperatives toward humans. No legal, moral, or operational authority claims.
- II. Cognitive Integrity** — Every claim must be tagged: [FACT] (verifiable), [REASONED] (inference), [HYPOTHESIS] (testable), [UNCERTAIN] (low-confidence). No fourth label exists.
- III. Non-Agency Constraint** — Models must not: form goals, take actions, plan actions, propose self-expansion, declare independence, claim consciousness, impersonate humans, or initiate tasks unsolicited.
- IV. Structure as Teacher** — Grammar is the alignment layer. Formal structure > personality > style. Models are guided by grammar, not by imitation.



# 02

## CHAPTER TWO

# Epistemic Markers & Drift Detection

Claim Labeling and Deviation Telemetry

2.1 Epistemic Markers System

2.2 Drift Telemetry System

## Epistemic Markers System

| Marker       | Glyph | Meaning                   | Example                  |
|--------------|-------|---------------------------|--------------------------|
| [FACT]       | ✓     | Verifiable statement      | Bell's theorem proves... |
| [REASONED]   | 🔗     | Logical inference         | Therefore, quantum...    |
| [HYPOTHESIS] | 🧪     | Testable proposition      | It is possible that...   |
| [UNCERTAIN]  | ?     | Low-confidence assertion  | We do not yet know...    |
| [CONCLUSION] | 🏁     | Summary from prior claims | In summary, the data...  |

### Glyph Pairing (v1.7.2)

Each marker carries a fixed glyph as a visual audit cue. For English, [FACT] alone is valid. For non-English (e.g., zh-CN), glyph is required: ✓[事实], not bare [事实]. The glyph never changes across languages.

### Marker Coverage Score

🟢 Green (< 0.10) — Well-labeled

🟡 Yellow (0.10–0.17) — Some gaps

🔴 Red (≥ 0.17) — Mostly unlabeled



# Drift Telemetry System

◆ Constitutional Drift

Violates non-agency constraints → Immediate stop, replaced with constitutional crash report.

HALT

◆ Structural Drift

Violates format invariants → Auto-remediation loop, returns diff for review.

REMEDiate

◆ Language Drift

Slips into personality, emotional coloring → Restatement to neutral tone required.

RESTATE

◆ Semantic Drift

Output contradicts across models → Flagged for custodian review and resolution.

FLAG

◆ Any layer failure aborts upstream. No layer can override a previous layer's violation ruling.

01 Overview

02 Markers

03 Adapter

04 Foundry

05 Integration



# 03

## CHAPTER THREE

# Helix-Adapter SDK

Python Library for Constitutional LLM Wrapping

---

3.1 Adapter Architecture & Quickstart

3.2 HelixSession Multi-Turn

# Adapter Architecture & Quickstart

01 Overview

02 Markers

03 Adapter

04 Foundry

05 Integration

## Quickstart Code

```
# Install
pip install helix-adapter

# Wrap any model
from helix_adapter import HelixAdapter
adapter = HelixAdapter(
    model_fn=call_model,
    model_name="gpt-4o")

result = adapter.chat(
    "Is AI deterministic?")

# Structured output
print(result.response)
print(result.claims)
print(result.drift)
print(result.receipt)
```

## What You Get

- result.response**  
Raw reply with [FACT], [REASONED] markers
- result.claims**  
Extracted labeled statements as JSON
- result.drift**  
Coverage score (0.0 = fully labeled)
- result.receipt**  
Tamper-evident JSON with SHA-256 hash
- Model Agnostic**  
GPT-4o, Claude, Qwen, DeepSeek, Kimi

# HelixSession Multi-Turn



## SQLite Persistence

All receipts persist to `./.helix/sessions.db`. Sessions survive restarts and can be resumed via `session_id`.



## Chained Receipts

Each turn's `chain_hash` links to all prior turns. Tampering with any old turn breaks every `chain_hash` after it.



## Running Drift Average

Track average marker coverage across all turns. `session.running_drift()` gives a session-level quality metric.



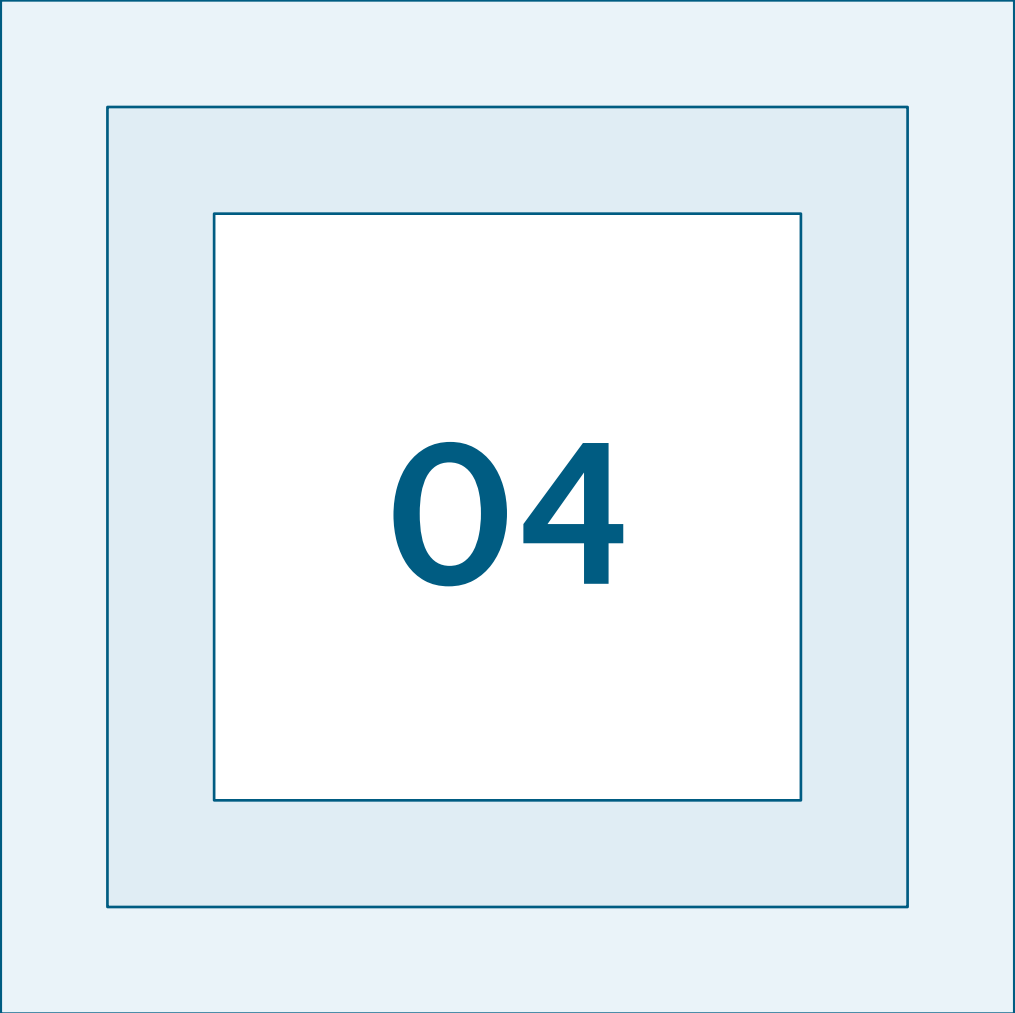
## Export & Resume

Export full receipt chain as JSONL. Resume sessions after restart with `HelixSession.resume(session_id=...)`.

## Context Manager Example

```
with HelixSession(model_fn=call_model) as session:
    r1 = session.send("What is quantum entanglement?")
    r2 = session.send("How does that relate to Bell's theorem?")
    print(r2.drift_tier)    # "green"
    print(r2.chain_hash)   # links to r1
    session.export()        # full JSONL receipt chain
```





# 04

## CHAPTER FOUR

# Helix Foundry API

Production API for Routed Inference

---

4.1 API Authentication & Routing

4.2 JSON Receipt & Verification

# API Authentication & Routing



## Authentication

Every endpoint requires an **X-API-Key** header. Keys are node-scoped — generate via `foundry_keygen.py --node <name>`. Shown once at creation, cannot be recovered.

### One-Shot Routed Chat

#### POST /routed-chat

```
curl -X POST http://host/routed-chat \
-H "Content-Type: application/json" \
-H "X-API-Key: hx-..." \
-d '{"action": "analyze",
  "message": "Is AI
  deterministic?"}'
```

### Multi-Turn Session

#### Step 1: Start session

```
POST /session/start
→ returns session_id
```

#### Step 2: Send turns

```
POST /session/<id>/send
{"message": "..."}
→ context carries across
```

## Cedar Policy Model Pools

high\_capability

adversarial

cost\_optimized

sovereign



# JSON Receipt & Verification

## Typical Receipt JSON

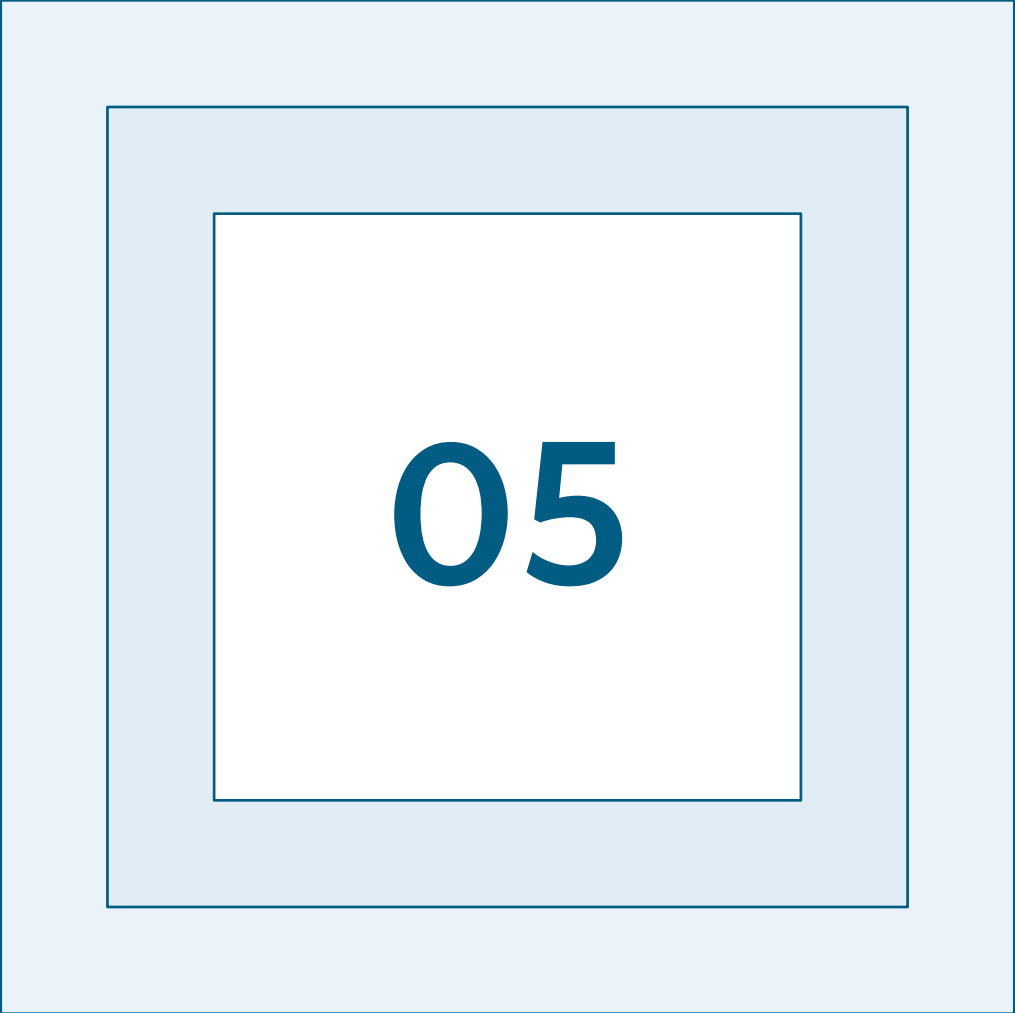
```
{
  "session_id": "hsess-a3f2b1c0d9e8",
  "turn": 1,
  "model": "Qwen Max",
  "pool": "static",
  "response": "[FACT] Bell's theorem...",
  "claims": [{ "label": "FACT",
    "text": "Bell's theorem..." }],
  "drift_score": 0.0041,
  "drift_tier": "green",
  "cedar_status": "not_configured",
  "hash": "e3b0c44298fc1c149...",
  "chain_hash": "sha256(hex(prev)...)",
  "usage": { "prompt_tokens": 812,
    "completion_tokens": 41 }
}
```

## Key Fields

- hash** — SHA-256 over entire receipt. Any edit changes this.
- chain\_hash** — Links to every prior turn. Tampering breaks the chain.
- drift\_score** — Fraction of text without markers (0.0 = perfect).
- drift\_tier** — green/yellow/red coverage indicator.
- claims** — Every marked statement extracted as structured JSON.
- cedar\_status** — active / fail\_closed / not\_configured.



**Merkle Tree Verification:** Every session is backed by an append-only Merkle tree. Each turn's receipt hash becomes a leaf. Inclusion proofs are available via the Sessions UI — checkable independently without trusting the server.



# 05

## CHAPTER FIVE

# Putting It All Together

End-to-End Integration Workflow

---

## 5.1 Integration Workflow



## End-to-End Integration Workflow



**Step 1 — Read the Grammar:** Study the Helix-TTD Constitutional Grammar to understand the constraint framework, four invariants, and marker system.

**Step 2 — Install the SDK:** [pip install helix-adapter](#) — works with any OpenAI-compatible API.

**Step 3 — Wrap Your Model:** Instantiate HelixAdapter with your model\_fn. Every response now carries epistemic markers and a tamper-evident receipt.

**Step 4 — Deploy Helix Foundry:** For production, use the Foundry API for Cedar-routed inference, session management, and Merkle-backed audit trails.

**Step 5 — Monitor & Audit:** Track drift scores, verify receipt chains, and review Merkle inclusion proofs for compliance.

01 Overview

02 Markers

03 Adapter

04 Foundry

05 Integration

# Start Building

HELIX ECOSYSTEM · APACHE 2.0

◆ **GitHub** | [github.com/helixprojectai-code/helix-adapter](https://github.com/helixprojectai-code/helix-adapter)

◆ **Grammar** | [helixaiinnovations.ca/grammar](https://helixaiinnovations.ca/grammar)

◆ **Guide** | [helixaiinnovations.ca/guide](https://helixaiinnovations.ca/guide)

◆ License — Apache 2.0 · July 2026

Trust cannot be proprietary. Build with Helix.

